

OPEN SOURCE AUDIT

Student Name	Ronit Mishra
Registration No.	24BSA10129
Course	Open Source Software
Chosen Software	Python Programming Language
Date of Submission	26-03-2026
Slot	B22

Introduction

1.1 What is Open Source Software?

At its heart **Open Source Software (OSS)** is built on the idea that software should be transparent. Instead of hiding the "secret sauce" (the source code) behind proprietary locks OSS makes it public. This allows anyone to inspect how it works, fix what is broken and share their improvements with the world. The term was born in 1998 when a group of developers—including Christine Peterson, Eric Raymond and Bruce Perens—realized that "free software" was a confusing term. In English "free" can mean "zero cost" (free beer) or "total liberty" (free speech). By rebranding it as **Open Source** they made the concept more approachable for the business world.

Today the **Open Source Initiative (OSI)** keeps watch over this ecosystem using ten specific criteria. These rules ensure that a project is not just "viewable" but truly collaborative—guaranteeing that users have the right to redistribute the code and build new things without being blocked by gatekeepers.

1.2 Why Open Source is important

Open source is not just a niche hobby; it is the scaffolding of the modern internet. It matters for a few very practical reasons:

- **It is Accessible:** Organizations can use world class tools without getting hit by massive licensing bills.
- **Security is a Team Sport:** When thousands of eyes can see the code bugs and vulnerabilities have nowhere to hide. They get found and fixed much faster than in **black box** proprietary systems.
- **Innovation via Collaboration:** Instead of reinventing the wheel developers build on top of each other's work. This "standing on the shoulders of giants" approach gave us the Linux kernel, the Apache web server and the Python language itself.
- **Ownership:** Open source prevents **vendor lock in**. You are not at the mercy of a single company's roadmap; you own your technological future.

1.3 1.3 Scope and Objectives of This Report

This audit takes a deep dive into **Python 3.x** which is the current gold standard maintained by the Python Software Foundation. We are not just looking at the code; we are looking at the entire "living" ecosystem. Our goals are to:

- **Trace the Roots:** Explore how Python's history and philosophy shaped the language we use today.
- **Decipher the Legal Side:** Break down Python's licensing and see how it stacks up against other open and closed models.
- **Under the Hood:** Document exactly how Python behaves when installed on a Linux system (specifically Debian/Ubuntu).
- **Map the Community:** Visualize the massive web of dependencies, downstream projects and community governance that keep Python thriving.

Part A — Origin and Philosophy

A1. Origin Story of Python

A1.1 Who Created Python and When

Python did not start in a corporate boardroom or a high tech lab. It actually began as a hobby project for a Dutch programmer named Guido van Rossum during his Christmas break in 1989. While working at a research institute in Amsterdam he decided to write an interpreter for a new scripting language because he was looking for something to keep him busy during the holidays. He gave it a name that had absolutely nothing to do with reptiles. Being a big fan of the British comedy show Monty Python's Flying Circus he chose Python because it felt irreverent and fun. He wanted to make sure that programming did not always have to be a dry or purely academic exercise. By February 1991 he shared the first public version with the world and it already had the features we use every day like functions and exception handling. As the years went by the language matured through several major milestones. Python 1.0 arrived in 1994 and by the time Python 2.0 showed up in 2000 the language had gained a lot of modern polish like list comprehensions and automatic garbage collection. Then came the big turning point in 2008 with the release of Python 3.0. This was a bold move because it was not backward compatible which meant developers had to eventually rewrite their old code to keep up. It was a necessary clean slate moment to fix old mistakes and get rid of redundant features. Even though it took a long time for everyone to migrate Python 2 finally reached its official end of life in January 2020. Now the Python Software Foundation focuses entirely on Python 3 as the stable and modern standard for the global developer community.

A1.2 The Problem Python Was Designed to Solve

Before Python came along the world of programming was a pretty punishing place. If you wanted to do anything serious you usually had to use languages like C or FORTRAN. These tools were incredibly fast but they were also exhausting to work with. They forced you to micromanage every single piece of computer memory and use syntax that was so wordy it was incredibly easy to make a mistake. Writing a simple script to read a file or move some data felt like building a house just to hang a picture frame. There was a massive gap between easy languages that could not do much and powerful languages that were far too difficult for most people to master. Guido van Rossum wanted to find a middle ground. He had worked on a teaching language called ABC which was wonderfully readable but it was a bit of a closed garden because it could not talk to other systems easily. He took the best parts of that user friendly design and combined them with the raw power of systems programming. The result was a language that prioritized human time over computer time. It allowed people to write programs in a fraction of the lines it would take in C. Suddenly automation and data science were accessible to everyone from researchers to students. Python essentially turned programming into a tool for creative problem solving instead of a constant battle with technical plumbing.

A2. License Analysis

A2.1 The Python Software Foundation License (PSFL)

Python is shared under the Python Software Foundation License which is a very friendly and permissive way to handle software. Since 2001 it has been officially OSI approved and it plays nicely with other big licenses like the GPL. Basically it gives you the right to use, copy and change

the code however you want. The only real rules are that you have to keep the original copyright notice and you cannot sue the foundation if something goes right or wrong. Unlike some other stricter licenses there is no copyleft rule here. This means that if you build something great using Python you are not forced to share your secret sauce with the rest of the world unless you truly want to.

A2.2 The Four Freedoms of Free Software

The Free Software Foundation, founded by Richard Stallman in 1985, defines free software through four essential freedoms:

1. Freedom 0: The freedom to run the program as you wish, for any purpose.
2. Freedom 1: The freedom to study how the program works, and change it so it does your computing as you wish. Access to source code is a precondition.
3. Freedom 2: The freedom to redistribute copies so you can help others.
4. Freedom 3: The freedom to distribute copies of your modified versions to others. By doing this you can give the whole community a chance to benefit from your changes.

A2.3 Difference between GPL and MIT

GPL and MIT are two different types of open source licenses:

GPL License:

- Requires that modified versions remain open source
- If you use GPL code in your software, your entire software must be open source
- Protects the freedom of the software strongly
- Sometimes called “copyleft”

MIT License:

- Allows people to use the code in proprietary software
- Does not require that modified versions be open source
- Gives more freedom to developers but less protection to the software
- Sometimes called “permissive”

A2.4 Can Companies Modify and Sell Python?

Because of this open approach companies are perfectly free to modify and even sell Python. This is a huge reason why tech giants like Google, Meta and Netflix have made it such a central part of their worlds. They can bake Python right into their products or tweak the underlying code to make it faster for their specific needs without being forced to reveal their proprietary secrets to competitors. This lack of legal friction has turned Python into a massive commercial success. While a stricter license ensures that every single improvement stays in the public eye it can stop big companies from adopting a tool in the first place. Python avoids that headache by letting businesses move fast without the legal baggage.

A2.5 Free as in Beer vs. Free as in Freedom

There is a famous saying in the tech community that helps explain the difference between being free as in beer and free as in freedom. Free as in beer just means it costs zero dollars which is great for your budget but does not actually give you any rights. Free as in freedom means you have the liberty to do what you want with the software regardless of the price. Python is one of those rare

and wonderful cases where you get both at the same time. It costs nothing to download and get started but it also respects your right to take it apart and rebuild it however you see fit. Many modern apps might be free to download today but they come with long contracts that stop you from truly owning the technology you use. Python stays far away from those kinds of traps.

A3. Ethics of Open Source Software

A3.1 Should All Software Be Open Source?

One of the most intense debates in the tech world is not about code but about values. Even though the open source movement and the free software movement look very similar on the surface they come from totally different places. Richard Stallman and the Free Software Foundation take what you might call an absolutist stance. To them all software should be free because restricting it is a moral failure. They compare it to hiding a recipe or a book from the public. They believe the freedom to share and change code is a basic human right for anyone using a computer. On the other side of the fence people like Eric Raymond and the Open Source Initiative take a more pragmatic approach. They do not necessarily care about the ethics of it all; they just think open source is a better way to build things. To them it is a superior methodology that leads to fewer bugs and better software. It is less about a moral crusade and more about what actually works in the real world.

A3.2 The Ethics of Corporate Open Source Use

A major point of tension lately involves big corporations that make massive profits off open source tools without giving much back to the community. Amazon Web Services is a famous example here. They built a multi billion dollar cloud empire using tools like Linux and Kubernetes that were built by volunteers. For a long time they were criticized for taking a lot and contributing very little in return. This "leeching" behavior has led some projects to change their rules. Projects like MongoDB and Elasticsearch adopted something called the Server Side Public License. This basically says that if a company wants to sell their software as a service they have to open source the entire system they use to run it. It is a way of forcing the big players to play fair. Python handles this a bit differently through the Python Software Foundation. This nonprofit manages the language and lets giants like Google, Microsoft and Meta pay for sponsorships. This gives the foundation the money it needs to keep Python running while letting the companies use the language without being forced to share every single line of their own private code.

A3.3 Open Source and Innovation

The proof is in the results: open source software moves much faster than anything kept behind a paywall. You can see this clearly in the world of machine learning. When researchers needed a way to do complex math they built NumPy. When big ideas like deep learning took off companies like Google and Meta released tools like TensorFlow and PyTorch for everyone to use. Because these tools are open a researcher in London can build on work done in Tokyo and share the results with someone in New York in a matter of minutes. Compare that to a proprietary tool like MATLAB. It is very powerful and popular in engineering but it is also very expensive and locked down. If you need a new feature that MATLAB does not have you have to wait for the company to build it which could take years. In the Python world if a feature is missing someone in the community usually builds it and puts it on the web for free by the next morning. This kind of global collaboration is simply impossible when the code is a secret.

Part B — Linux Footprint

B1. Installation on Debian/Ubuntu Linux

B1.1 Installation Methods

Python can be installed on Debian/Ubuntu systems through several methods, each suited to different use cases:

Method 1: System Package Manager (Recommended for system use)

```
sudo apt update
sudo apt install python3 python3-pip python3-venv
```

B1.2 File System Locations

After installation, Python's files are distributed across the Linux filesystem in accordance with the Filesystem Hierarchy Standard (FHS):

Main Binary	/usr/bin/python3
Versioned Binary	/usr/bin/python3.12 (symlinked from python3)
Standard Library	/usr/lib/python3.12/
System-wide Packages	/usr/lib/python3/dist-packages/
User Packages (pip)	~/.local/lib/python3.12/site-packages/
Virtual Env Packages	[venv]/lib/python3.12/site-packages/
Config Files	/etc/python3/
pip Config	~/.config/pip/pip.conf
Man Pages	/usr/share/man/man1/python3.1.gz

B1.3 Verification Commands

After installation, the following commands verify Python's correct installation and system integration:

```
python3 --version          # Output: Python 3.12.x
which python3              # Output: /usr/bin/python3
python3 -c "import sys; print(sys.path)"  # Show module search path
pip3 --version             # pip 23.x from /usr/lib/python3
python3 -m site            # Show site-packages directories
dpkg -L python3            # List all files installed by the python3
package
```

B1.4 User and Permission Model

Python runs with your own personal permissions. You do not need special administrative powers for regular work or for putting packages into your own user specific folders. This design follows the idea of least privilege which is a big part of how Unix systems stay secure. When you install it through the system tools the files belong to the root user but anyone can read them. It is important to note that the Python binary is not setuid root; it actually runs with your own authority to keep the system safe. If you use the user flag with pip your packages land in a local folder and you never have to worry about needing a password to get things moving.

Virtual environments are the real stars when it comes to keeping things organized. A virtual environment creates a private little island for your Python installation inside any folder you choose. It comes with its own set of packages along with its own version of the installer and the main binary. This is great because it lets different projects use different versions of tools without any fighting or messy conflicts. Best of all it keeps everything contained so you can manage your work without ever needing to touch the sensitive parts of the operating system.

```
python3 -m venv myproject/env
source myproject/env/bin/activate
pip install requests          # Installs into env, no sudo needed
```

B1.5 Service Management

Python itself is not a daemon or service — it is an interpreter that executes scripts and exits. However, Python applications often run as long-running services managed by systemd. A typical systemd unit file for a Python web application (e.g., a Django app) would look like:

```
[Unit]
Description=Django Web Application
After=network.target

[Service]
User=www-data
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/env/bin/python manage.py runserver
Restart=always

[Install]
WantedBy=multi-user.target
```

B1.6 Updates and Maintenance

Python on Debian/Ubuntu is updated through the standard apt package management system. System Python updates are managed by the distribution and tied to the distribution's release cycle, which means some distributions may lag behind the latest upstream Python version.

```
sudo apt update && sudo apt upgrade python3    # System update
pip3 install --upgrade pip                     # Upgrade pip itself
pip3 list --outdated                           # Check outdated packages
pip3 install --upgrade package_name            # Upgrade a package
```

Part C — The FOSS Ecosystem

C1. Dependencies and Runtime Environment

Python depends on:

- C libraries
- GCC compiler
- System libraries

C2. Major Tools and Frameworks Built on Python

Tools Built on Python

- Django for web development
- NumPy for data analysis
- TensorFlow for machine learning

C3. Community and Governance

Python is maintained by the Python Software Foundation. It has a large global community with active contributors. Role in Development.

Python is widely used in:

- Web development
- Data science
- Automation
- Artificial intelligence

Part D — Comparative Analysis

D1. Python vs. Proprietary and Other Open Source Alternatives

The following table provides a comprehensive comparison of Python against MATLAB

Feature	Python (Open Source)	MATLAB (Proprietary)
Cost	Free (no license fee)	Expensive (~\$2,000+/yr)
Source Code	Fully open (GitHub)	Closed/proprietary
Flexibility	Very high	Limited to toolboxes
Community Size	Extremely large (millions)	Smaller (mostly academia)
Modification	Allowed freely	Not allowed
License	PSF (permissive)	Proprietary EULA
Primary Use	General purpose, AI, web	Engineering, simulation
Package Ecosystem	PyPI (500,000+ packages)	MATLAB toolboxes (paid)
Learning Curve	Beginner-friendly	Moderate (domain-specific)
Industry Adoption	AI, web, finance, science	Academia, engineering

D2. Detailed Analysis by Domain

D2.1 Scientific Computing: Python vs MATLAB

MATLAB, or Matrix Laboratory, is created by MathWorks. It is one of the few areas where proprietary software has consistently outperformed open-source software. The development environment, documentation, and specific toolboxes for control systems, signal processing, etc., made MATLAB the de facto standard for engineering and applied mathematics for decades. However, the gap between MATLAB and open-source tools has been bridged by the development of the Python scientific stack. NumPy has array computations similar to MATLAB's native support for matrix computations. SciPy has most of the functionality of MATLAB's base toolbox. The only areas where MATLAB has significant advantages are its Simulink block-diagram simulation environment, which has no open-source equivalent, its interactive model-based design tools, and its deep penetration into some engineering courses and corporate environments.

The advantages of using Python over MATLAB are significant. The cost of using MATLAB is zero, whereas the cost of a MATLAB license for one seat is between \$2,000 and \$5,000. The number of developers supporting and using Python is vastly higher. Also, the ability to deploy Python code in production web/cloud environments without any further licensing issues makes it the better choice for most tasks.

D3. Total Cost of Ownership Analysis

When considering other aspects of the software beyond the licensing fees, the total cost of ownership (TCO), which includes training costs, support costs, infrastructure costs, and integration costs, will also be in favor of Python. This is because:

- License Cost: Python has zero license costs. MATLAB has a license cost of \$2,000 to \$5,000+ per seat per year.
- Training Cost: Python has the largest number of trained professionals in the world. MATLAB professionals are highly paid.
- Support Cost: Python has commercial support from Anaconda, Enthought, and Databricks. It also has a massive number of free support providers. MATLAB has support only from MathWorks.
- Deployment Cost: Python can be deployed for free to any server or any device. MATLAB requires license validation to be done.
- Integration Cost: Python can be integrated with almost any other technology. MATLAB has very few integrations.

Shell Scripts — Linux Audit Scripts

Script 1 — System Identity (Script1.sh)

```
ronit@DESKTOP-REA10UU:/mnt/c/Users/acer/OneDrive/Scripts_OSS$ chmod +x *.sh
./Script1.sh
=====
Open Source Audit - Ronit Mishra
=====
Software : Python
Kernel   : 6.6.87.2-microsoft-standard-WSL2
User     : ronit
Uptime   : up 14 minutes
Date     : Thu Mar 26 14:37:52 UTC 2026
Distro   : "Ubuntu 24.04.4 LTS"
License  : GNU General Public License (Linux Kernel)
```

Script 2 — Package Inspector (Script2.sh)

```
ronit@DESKTOP-REA10UU:/mnt/c/Users/acer/OneDrive/Scripts_OSS$ ./Script2.sh
python3 is installed.
ii  libpython3-stdlib:amd64      3.12.3-0ubuntu2.1      amd64      interactive high-level object-oriented language (default python3 vers
ion)
Python: simple, powerful open-source programming language
```

Script 3 — Disk Auditor (Script3.sh)

```
ronit@DESKTOP-REA10UU:/mnt/c/Users/acer/OneDrive/Scripts_OSS$ ./Script3.sh
Directory Audit Report
-----
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 59M
/home => Permissions: drwxr-xr-x root root | Size: 28K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 24K
Python config directory found:
drwxr-xr-x 2 root root 4096 Feb 10 00:55 /etc/python3
```

Script 4 — Log Analyzer (Script4.sh)

```
ronit@DESKTOP-REA10UU:/mnt/c/Users/acer/OneDrive/Scripts_OSS$ ./Script4.sh
Usage: ./Script4.sh <logfile> [keyword]
```

Script 5 — Manifesto Generator (Script5.sh)

```
ronit@DESKTOP-REA10UU:/mnt/c/Users/acer/OneDrive/Scripts_OSS$ ./Script5.sh
Answer three questions:
1. Tool you use: python
2. Freedom means: independence
3. You want to build: an open-source AI assistant
Manifesto saved to manifesto_ronit.txt
On 26 March 2026, I believe open source tools like python represent independence.
I want to build an open-source AI assistant and share it freely with the world.
```

Conclusion

Summary of Findings

This audit of the Python project as an FOSS project has revealed some key points. The first is the fundamental nature of the open-source approach of the Python project. The open-source nature of the project was not an accident, but the deliberate choice of its creator, Guido van Rossum, in 1991, which has resulted in the development of the powerful and widely used programming tool it has become today.

The second key point is the balance between open-source and commercial viability, as the project is licensed under the PSFL. The PSFL, being a permissive license, has allowed the project to achieve the balance between open-source and commercial viability, while allowing the open-source community to use, study, modify, and distribute the project.

Third, Python's footprint on Linux is a reflection of how well it integrates into operating systems and operating system-level thinking. Its compliance with the Filesystem Hierarchy Standard, support for virtual environments, and pip package management toolset all combine to give developers a powerful and well-thought-out toolset.

Fourth, the FOSS ecosystem built around Python is one of a kind in terms of depth and breadth. From Django to TensorFlow to Ansible, the software built atop Python is worth billions of dollars in economic value and is available to all for no cost.

Fifth, comparative analysis confirms Python's advantages over proprietary alternatives across virtually every dimension except in highly specialized niches (MATLAB's Simulink, R's statistical depth). The total cost of ownership strongly favors Python in most professional and academic contexts.

End of Report — Ronit Mishra | 24BSA10129 | Open Source Software